

torch.linalg.matrix_rank#

https://pytorch.org/docs/stable/generated/torch.linalg.matrix_rank.html

Description:

Computes the numerical rank of a matrix. The matrix rank is computed as the number of singular values (or eigenvalues in absolute value when `hermitian= True`) that are greater than $\max(\text{atol}, \sigma_1 * \text{rtol})$ threshold, where σ_1 is the largest singular value (or eigenvalue). Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if `A` is a batch of matrices then the output has the same batch dimensions. If `hermitian= True`, `A` is assumed to be Hermitian if complex or symmetric if real, but this is not checked internally. Instead, just the lower triangular part of the matrix is used in the computations. If `rtol` is not specified and `A` is a matrix of dimensions (m, n) , the relative tolerance is set to be $\text{rtol} = \max(m, n) \epsilon$ and ϵ is the epsilon value for the dtype of `A` (see `finfo`). If `rtol` is not specified and `atol` is specified to be larger than zero then `rtol` is set to zero.

Function Signature

```
torch.linalg.matrix_rank(A, *, atol=None, rtol=None,  
hermitian=False, out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

`atol` (float, Tensor, optional) – the absolute tolerance value. When `None` it's considered to be zero. Default: `None`. `rtol` (float, Tensor, optional) – the relative tolerance value. See above for the value it takes when `None`. Default: `None`. `hermitian` (bool) – indicates whether `A` is Hermitian if complex or symmetric if real. Default: `False`. `out` (Tensor, optional) – output tensor. Ignored if `None`. Default: `None`.

Code Examples

```
>>> A = torch.eye(10)
>>> torch.linalg.matrix_rank(A)
tensor(10)
>>> B = torch.eye(10)
>>> B[0, 0] = 0
>>> torch.linalg.matrix_rank(B)
tensor(9)

>>> A = torch.randn(4, 3, 2)
>>> torch.linalg.matrix_rank(A)
tensor([2, 2, 2, 2])

>>> A = torch.randn(2, 4, 2, 3)
>>> torch.linalg.matrix_rank(A)
tensor([[2, 2, 2, 2],
        [2, 2, 2, 2]])

>>> A = torch.randn(2, 4, 3, 3, dtype=torch.complex64)
>>> torch.linalg.matrix_rank(A)
tensor([[3, 3, 3, 3],
        [3, 3, 3, 3]])
>>> torch.linalg.matrix_rank(A, hermitian=True)
tensor([[3, 3, 3, 3],
        [3, 3, 3, 3]])
>>> torch.linalg.matrix_rank(A, atol=1.0, rtol=0.0)
tensor([[3, 2, 2, 2],
        [1, 2, 1, 2]])
>>> torch.linalg.matrix_rank(A, atol=1.0, rtol=0.0,
hermitian=True)
tensor([[2, 2, 2, 1],
        [1, 2, 2, 2]])
```

Notes & Warnings

NOTE

Note This function has NumPy compatible variant `linalg.matrix_rank(A, tol, hermitian=False)`. However, use of the positional argument `tol` is deprecated in favor of `atol` and `rtol`.

NOTE

Note The matrix rank is computed using a singular value decomposition `torch.linalg.svdvals()` if `hermitian= False` (default) and the eigenvalue decomposition `torch.linalg.eigvalsh()` when `hermitian= True`. When inputs are on a CUDA device, this function synchronizes that device with the CPU.

Detailed Documentation

Documentation

Computes the numerical rank of a matrix.

The matrix rank is computed as the number of singular values (or eigenvalues in absolute value when `hermitian= True`) that are greater than $\max(\text{atol}, \sigma_1 * \text{rtol})$ threshold, where σ_1 is the largest singular value (or eigenvalue).

Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if `A` is a batch of matrices then the output has the same batch

dimensions.

If `hermitian= True`, `A` is assumed to be Hermitian if complex or symmetric if real, but this is not checked internally. Instead, just the lower triangular part of the matrix is used in the computations.

If `rtol` is not specified and `A` is a matrix of dimensions (m, n) , the relative tolerance is set to be $\text{rtol} = \max(m, n) \epsilon$ and ϵ is the epsilon value for the dtype of `A` (see `finfo`). If `rtol` is not specified and `atol` is specified to be larger than zero then `rtol` is set to zero.

If `atol` or `rtol` is a `torch.Tensor`, its shape must be broadcastable to that of the singular values of `A` as returned by `torch.linalg.svdvals()`.

This function has NumPy compatible variant `linalg.matrix_rank(A, tol, hermitian=False)`. However, use of the positional argument `tol` is deprecated in favor of `atol` and `rtol`.

The matrix rank is computed using a singular value decomposition `torch.linalg.svdvals()` if `hermitian= False` (default) and the eigenvalue decomposition `torch.linalg.eigvalsh()` when `hermitian= True`. When inputs are on a CUDA device, this function synchronizes that device with the CPU.

`A (Tensor)` – tensor of shape $(*, m, n)$ where $*$ is zero or more batch dimensions.

`tol (float, Tensor, optional)` – [NumPy Compat] Alias for `atol`. Default: `None`.

`atol (float, Tensor, optional)` – the absolute tolerance value. When `None` it's considered to be zero. Default: `None`.

`rtol (float, Tensor, optional)` – the relative tolerance value. See above for the value it takes when `None`. Default: `None`.

hermitian (bool) – indicates whether A is Hermitian if complex or symmetric if real.
Default: False.

out (Tensor, optional) – output tensor. Ignored if None. Default: None.